
GUIDE 07

Deploy Agentic AI Apps to Google Cloud (GCP)

Cloud Run, GKE, Artifact Registry, and Vertex AI — step by step

Agentic AI Builder Expert Bootcamp

Batch 4.0 | April 2026

Table of Contents

1. GCP Overview for AI Deployments
2. Prerequisites and CLI Setup
3. Building Production Dockerfiles
4. Google Artifact Registry Setup
5. Building and Pushing Images
6. Deploying to Cloud Run (Recommended)
7. Deploying to Google Kubernetes Engine (GKE)
8. Deploying to Compute Engine (VM)
9. Using Vertex AI Endpoints
10. Secret Management with Secret Manager
11. CI/CD with Cloud Build
12. Networking and Custom Domains
13. Monitoring with Cloud Operations
14. Common Errors and Fixes
15. GCP Deployment Options Comparison

1. GCP Overview for AI Deployments

Google Cloud Platform provides multiple deployment targets for agentic AI applications, each suited to different scale and complexity requirements.

Key Services for Agent Deployments

- **Cloud Run:** Fully managed serverless containers. Auto-scales to zero. Best for most agent APIs.
- **Google Kubernetes Engine (GKE):** Managed Kubernetes for complex multi-agent architectures.
- **Compute Engine:** Full VMs for maximum control. Good for GPU workloads or persistent processes.
- **Vertex AI:** Google's ML platform with model endpoints, pipelines, and model monitoring.
- **Artifact Registry:** Docker container registry (replaces Container Registry).
- **Secret Manager:** Secure storage for API keys and credentials.
- **Cloud Build:** CI/CD pipeline for automated builds and deployments.

2. Prerequisites and CLI Setup

2.1 Install Google Cloud CLI

```
# macOS
brew install google-cloud-sdk

# Windows
# Download from: https://cloud.google.com/sdk/docs/install

# Linux (Debian/Ubuntu)
sudo apt-get install apt-transport-https ca-certificates gnupg curl
curl https://packages.cloud.google.com/apt/doc/apt-key.gpg | \
  sudo gpg --dearmor -o /usr/share/keyrings/cloud.google.gpg
echo "deb [signed-by=/usr/share/keyrings/cloud.google.gpg] \
  https://packages.cloud.google.com/apt cloud-sdk main" | \
  sudo tee /etc/apt/sources.list.d/google-cloud-sdk.list
sudo apt-get update && sudo apt-get install google-cloud-cli
```

2.2 Authenticate and Configure

```
# Login
gcloud auth login

# Set project
gcloud config set project YOUR_PROJECT_ID

# Set default region (Singapore)
gcloud config set run/region asia-southeast1
gcloud config set compute/zone asia-southeast1-a

# Enable required APIs
gcloud services enable \
  run.googleapis.com \
```

```
artifactregistry.googleapis.com \  
cloudbuild.googleapis.com \  
secretmanager.googleapis.com \  
container.googleapis.com  
  
# Verify  
gcloud config list
```

2.3 Install Docker

```
# Verify Docker is installed  
docker --version  
  
# Configure Docker for Artifact Registry  
gcloud auth configure-docker asia-southeast1-docker.pkg.dev
```

3. Building Production Dockerfiles

3.1 Optimized Multi-Stage Dockerfile

```
# Dockerfile  
# Stage 1: Install dependencies  
FROM python:3.11-slim AS builder  
WORKDIR /build  
COPY requirements.txt .  
RUN pip install --no-cache-dir --prefix=/install \  
    -r requirements.txt  
  
# Stage 2: Production image  
FROM python:3.11-slim  
WORKDIR /app  
  
# Copy installed packages  
COPY --from=builder /install /usr/local  
  
# Copy application  
COPY . .  
  
# Create non-root user (required by Cloud Run)  
RUN useradd --create-home appuser  
USER appuser  
  
# Cloud Run uses PORT env var (defaults to 8080)  
ENV PORT=8080  
EXPOSE 8080  
  
CMD exec uvicorn api.main:app \  
    --host 0.0.0.0 \  
    --port $PORT \  
    --workers 1
```

Warning: Cloud Run sets the `PORT` environment variable (default 8080, not 8000). Your application **MUST** read `PORT` from the environment. This is the #1 cause of deployment failures on Cloud Run.

3.2 FastAPI Server for Cloud Run

```
# api/main.py
import os
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from langchain_core.messages import HumanMessage
from agent.graph import graph

app = FastAPI(title="LangGraph Agent - GCP")

class ChatRequest(BaseModel):
    message: str
    thread_id: str = "default"

class ChatResponse(BaseModel):
    response: str
    steps: int

@app.get("/")
def root():
    return {"status": "healthy", "platform": "gcp-cloud-run"}

@app.post("/chat", response_model=ChatResponse)
async def chat(req: ChatRequest):
    try:
        result = graph.invoke({
            "messages": [HumanMessage(content=req.message)]
        })
        return ChatResponse(
            response=result["messages"][-1].content,
            steps=len(result["messages"])
        )
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
    import uvicorn
    port = int(os.environ.get("PORT", 8080))
    uvicorn.run(app, host="0.0.0.0", port=port)
```

3.3 .dockerignore

```
.git
.gitignore
__pycache__
*.pyc
.env
.venv
venv/
tests/
docs/
*.md
.pytest_cache
.mypy_cache
.DS_Store
Dockerfile
```

```
docker-compose.yml
```

4. Google Artifact Registry Setup

Artifact Registry is Google's recommended container registry, replacing the legacy Container Registry.

Step 1: Create a Repository

```
gcloud artifacts repositories create agentic-ai-repo \
  --repository-format=docker \
  --location=asia-southeast1 \
  --description="Agentic AI container images"

# Verify
gcloud artifacts repositories list \
  --location=asia-southeast1
```

Step 2: Configure Docker Authentication

```
gcloud auth configure-docker \
  asia-southeast1-docker.pkg.dev

# This updates ~/.docker/config.json automatically
```

***Tip:** Use regional repositories (asia-southeast1) instead of multi-region (us, eu) for faster pulls from Cloud Run instances in the same region.*

5. Building and Pushing Images

5.1 Local Build and Push

```
# Define variables
PROJECT_ID=$(gcloud config get-value project)
REGION=asia-southeast1
REPO=agentic-ai-repo
IMAGE=langgraph-agent

# Full image path
IMAGE_URI=${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO}/${IMAGE}

# Build
docker build -t ${IMAGE_URI}:v1.0 .
docker tag ${IMAGE_URI}:v1.0 ${IMAGE_URI}:latest

# Push
docker push ${IMAGE_URI}:v1.0
docker push ${IMAGE_URI}:latest

# Verify
gcloud artifacts docker images list \
  ${REGION}-docker.pkg.dev/${PROJECT_ID}/${REPO}
```

5.2 Cloud Build (Remote Build)

For CI/CD or when you don't want to build locally, use Cloud Build to build directly in GCP.

```
# Build remotely using Cloud Build
gcloud builds submit --tag ${IMAGE_URI}:v1.0

# Or use a cloudbuild.yaml for complex builds
# (See Section 11 for full CI/CD configuration)
```

6. Deploying to Cloud Run (Recommended)

Cloud Run is the recommended deployment target for most agentic AI applications. It provides serverless, auto-scaling containers with scale-to-zero capability.

6.1 Deploy the Service

Step 1: Deploy from Artifact Registry

```
gcloud run deploy langgraph-agent \
  --image=${IMAGE_URI}:v1.0 \
  --region=asia-southeast1 \
  --platform=managed \
  --port=8080 \
  --memory=2Gi \
  --cpu=2 \
  --timeout=300 \
  --concurrency=10 \
  --min-instances=0 \
  --max-instances=10 \
  --allow-unauthenticated \
  --set-env-vars="LANGCHAIN_TRACING_V2=true" \
  --set-secrets="OPENAI_API_KEY=openai-key:latest"

# Note: --set-secrets requires Secret Manager setup (Section 10)
```

Step 2: Get the Service URL

```
gcloud run services describe langgraph-agent \
  --region=asia-southeast1 \
  --format="value(status.url)"

# Output: https://langgraph-agent-xxxxx-as.a.run.app
```

Step 3: Test the Deployment

```
# Health check
curl https://langgraph-agent-xxxxx-as.a.run.app/

# Test chat endpoint
curl -X POST https://langgraph-agent-xxxxx-as.a.run.app/chat \
  -H "Content-Type: application/json" \
  -d '{"message": "Hello, agent!"}'
```

6.2 Cloud Run Configuration Best Practices

Setting	Recommended Value	Why
--memory	2Gi (minimum for agents)	LangGraph + LLM libraries need memory headroom
--cpu	2	Agent chains benefit from parallel processing
--timeout	300 (5 minutes)	Multi-step agent chains can take time
--concurrency	10	Each agent request is heavy; limit concurrent requests
--min-instances	0 (dev) / 1 (prod)	Scale to zero saves money; keep 1 warm for low latency
--max-instances	10-50	Prevent runaway scaling and cost spikes
--cpu-boost	enabled	Faster cold starts (reduces startup from 10s to 2-3s)

6.3 Update a Deployment

```
# Deploy new version
gcloud run deploy langgraph-agent \
  --image=${IMAGE_URI}:v2.0 \
  --region=asia-southeast1

# Traffic splitting (canary deployment)
gcloud run services update-traffic langgraph-agent \
  --region=asia-southeast1 \
  --to-revisions=langgraph-agent-v2=10 \
  --to-revisions=langgraph-agent-v1=90

# Rollback to previous revision
gcloud run services update-traffic langgraph-agent \
  --region=asia-southeast1 \
  --to-latest
```

Tip: Cloud Run's traffic splitting lets you do canary deployments. Route 10% traffic to a new prompt version and monitor before full rollout.

7. Deploying to Google Kubernetes Engine (GKE)

GKE is ideal for complex multi-agent architectures requiring custom networking, service meshes, GPU nodes, or fine-grained resource control.

Step 1: Create a GKE Cluster


```
# Autopilot cluster (recommended - Google manages nodes)
gcloud container clusters create-auto agentic-ai-cluster \
  --region=asia-southeast1 \
  --release-channel=stable

# Standard cluster (you manage nodes)
gcloud container clusters create agentic-ai-cluster \
  --region=asia-southeast1 \
  --num-nodes=2 \
  --machine-type=e2-standard-4 \
  --enable-autoscaling \
  --min-nodes=1 \
  --max-nodes=5

# Get credentials
gcloud container clusters get-credentials agentic-ai-cluster \
  --region=asia-southeast1
```

Step 2: Create Kubernetes Manifests

```
# k8s/namespace.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: agentic-ai

---
# k8s/deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: langgraph-agent
  namespace: agentic-ai
  labels:
    app: langgraph-agent
spec:
  replicas: 2
  selector:
    matchLabels:
      app: langgraph-agent
  template:
    metadata:
      labels:
        app: langgraph-agent
    spec:
      serviceAccountName: agent-sa
      containers:
        - name: agent
          image: asia-southeast1-docker.pkg.dev/PROJECT/agentic-ai-repo/langgraph-agent:v1.0
          ports:
            - containerPort: 8080
          env:
            - name: PORT
              value: "8080"
            - name: OPENAI_API_KEY
              valueFrom:
                secretKeyRef:
```

```

        name: agent-secrets
        key: openai-api-key
    resources:
        requests:
            cpu: "500m"
            memory: "1Gi"
        limits:
            cpu: "2000m"
            memory: "4Gi"
    livenessProbe:
        httpGet:
            path: /
            port: 8080
        initialDelaySeconds: 15
        periodSeconds: 30
    readinessProbe:
        httpGet:
            path: /
            port: 8080
        initialDelaySeconds: 10
        periodSeconds: 10
---
# k8s/service.yaml
apiVersion: v1
kind: Service
metadata:
    name: langgraph-agent-svc
    namespace: agentic-ai
spec:
    type: LoadBalancer
    ports:
        - port: 80
          targetPort: 8080
          protocol: TCP
    selector:
        app: langgraph-agent
---
# k8s/hpa.yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
    name: langgraph-agent-hpa
    namespace: agentic-ai
spec:
    scaleTargetRef:
        apiVersion: apps/v1
        kind: Deployment
        name: langgraph-agent
    minReplicas: 2
    maxReplicas: 20
    metrics:
        - type: Resource
          resource:
            name: cpu
            target:
                type: Utilization

```

```
averageUtilization: 70
```

Step 3: Deploy and Verify

```
kubectl apply -f k8s/namespace.yaml
kubectl apply -f k8s/deployment.yaml
kubectl apply -f k8s/service.yaml
kubectl apply -f k8s/hpa.yaml

# Check status
kubectl get pods -n agentic-ai
kubectl get svc -n agentic-ai
kubectl get hpa -n agentic-ai

# Get external IP
kubectl get svc langgraph-agent-svc -n agentic-ai \
  -o jsonpath='{.status.loadBalancer.ingress[0].ip}'
```

8. Deploying to Compute Engine (VM)

Compute Engine gives you full VM control. Use this for GPU workloads, persistent processes, or when you need direct SSH access.

Step 1: Create a VM Instance

```
gcloud compute instances create agent-vm \
  --zone=asia-southeast1-a \
  --machine-type=e2-standard-4 \
  --image-family=ubuntu-2404-lts-amd64 \
  --image-project=ubuntu-os-cloud \
  --boot-disk-size=50GB \
  --tags=http-server \
  --metadata=startup-script='#!/bin/bash
apt-get update
apt-get install -y docker.io docker-compose-plugin
systemctl enable docker
systemctl start docker'
```

Step 2: Open Firewall for HTTP

```
gcloud compute firewall-rules create allow-agent-http \
  --direction=INGRESS \
  --priority=1000 \
  --network=default \
  --action=ALLOW \
  --rules=tcp:8080 \
  --target-tags=http-server
```

Step 3: SSH and Deploy

```
# SSH into the VM
gcloud compute ssh agent-vm --zone=asia-southeast1-a

# Inside the VM:
```

```
gcloud auth configure-docker asia-southeast1-docker.pkg.dev
docker pull asia-southeast1-docker.pkg.dev/PROJECT/agent-ai-repo/langgraph-agent:v1.0

docker run -d \
  --name agent \
  --restart=always \
  -p 8080:8080 \
  -e OPENAI_API_KEY="sk-xxx" \
  asia-southeast1-docker.pkg.dev/PROJECT/agent-ai-repo/langgraph-agent:v1.0
```

9. Using Vertex AI Endpoints

Vertex AI provides managed model serving with built-in monitoring, A/B testing, and auto-scaling. This is useful when your agent wraps a custom fine-tuned model.

```
# Build a custom prediction container
# Vertex AI expects specific health/prediction endpoints

# Dockerfile.vertex
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .

# Vertex AI required env vars
ENV AIP_HTTP_PORT=8080
ENV AIP_HEALTH_ROUTE=/health
ENV AIP_PREDICT_ROUTE=/predict

CMD ["uvicorn", "api.vertex_main:app", "--host", "0.0.0.0", "--port", "8080"]

# api/vertex_main.py
from fastapi import FastAPI
from pydantic import BaseModel
from agent.graph import graph
from langchain_core.messages import HumanMessage

app = FastAPI()

class PredictRequest(BaseModel):
    instances: list

class PredictResponse(BaseModel):
    predictions: list

@app.get("/health")
def health():
    return {"status": "healthy"}

@app.post("/predict", response_model=PredictResponse)
async def predict(req: PredictRequest):
    predictions = []
    for instance in req.instances:
```

```

        result = graph.invoke({
            "messages": [HumanMessage(content=instance["message"])]
        })
        predictions.append({
            "response": result["messages"][-1].content
        })
    return PredictResponse(predictions=predictions)

# Upload model and create endpoint
gcloud ai models upload \
    --region=asia-southeast1 \
    --display-name=langgraph-agent \
    --container-image-uri=${IMAGE_URI}:v1.0 \
    --container-health-route=/health \
    --container-predict-route=/predict \
    --container-ports=8080

gcloud ai endpoints create \
    --region=asia-southeast1 \
    --display-name=agent-endpoint

# Deploy model to endpoint
gcloud ai endpoints deploy-model ENDPOINT_ID \
    --region=asia-southeast1 \
    --model=MODEL_ID \
    --display-name=agent-v1 \
    --machine-type=e2-standard-4 \
    --min-replica-count=1 \
    --max-replica-count=5 \
    --traffic-split=0=100

```

10. Secret Management with Secret Manager

Step 1: Create Secrets

```

# Create a secret
echo -n "sk-your-openai-key" | \
    gcloud secrets create openai-key \
        --data-file=- \
        --replication-policy=automatic

echo -n "ls-your-langsmith-key" | \
    gcloud secrets create langsmith-key \
        --data-file=- \
        --replication-policy=automatic

# List secrets
gcloud secrets list

```

Step 2: Grant Access to Cloud Run

```

# Get the Cloud Run service account
SA=$(gcloud run services describe langgraph-agent \
    --region=asia-southeast1 \

```

```

--format="value(spec.template.spec.serviceAccountName)")

# If default:
SA=$(gcloud iam service-accounts list \
  --filter="displayName:Compute Engine" \
  --format="value(email)")

# Grant access
gcloud secrets add-iam-policy-binding openai-key \
  --member="serviceAccount:${SA}" \
  --role="roles/secretmanager.secretAccessor"

gcloud secrets add-iam-policy-binding langsmith-key \
  --member="serviceAccount:${SA}" \
  --role="roles/secretmanager.secretAccessor"

```

Step 3: Use Secrets in Cloud Run

```

gcloud run services update langgraph-agent \
  --region=asia-southeast1 \
  --set-secrets="OPENAI_API_KEY=openai-key:latest,\
LANGCHAIN_API_KEY=langsmith-key:latest"

```

11. CI/CD with Cloud Build

```

# cloudbuild.yaml
steps:
  # Step 1: Build the Docker image
  - name: 'gcr.io/cloud-builders/docker'
    args:
      - 'build'
      - '-t'
      - 'asia-southeast1-docker.pkg.dev/$PROJECT_ID/agent-ai-repo/langgraph-agent:$SHORT_SHA'
      - '-t'
      - 'asia-southeast1-docker.pkg.dev/$PROJECT_ID/agent-ai-repo/langgraph-agent:latest'
      - '.'

  # Step 2: Push images
  - name: 'gcr.io/cloud-builders/docker'
    args:
      - 'push'
      - '--all-tags'
      - 'asia-southeast1-docker.pkg.dev/$PROJECT_ID/agent-ai-repo/langgraph-agent'

  # Step 3: Deploy to Cloud Run
  - name: 'gcr.io/google.com/cloudsdktool/cloud-sdk'
    entrypoint: gcloud
    args:
      - 'run'
      - 'deploy'
      - 'langgraph-agent'
      - '--image=asia-southeast1-docker.pkg.dev/$PROJECT_ID/agent-ai-repo/langgraph-agent:$SHORT_SHA'

```

```

- '--region=asia-southeast1'
- '--platform=managed'

options:
  logging: CLOUD_LOGGING_ONLY

# Trigger: auto-deploy on push to main
trigger:
  branch: main

# Set up the trigger
gcloud builds triggers create github \
  --name=deploy-agent \
  --repo-name=langgraph-agent \
  --repo-owner=YOUR_GITHUB_ORG \
  --branch-pattern='^main$' \
  --build-config=cloudbuild.yaml

```

12. Networking and Custom Domains

12.1 Map a Custom Domain to Cloud Run

```

# Map domain
gcloud run domain-mappings create \
  --service=langgraph-agent \
  --domain=agent.yourdomain.com \
  --region=asia-southeast1

# Get DNS records to configure
gcloud run domain-mappings describe \
  --domain=agent.yourdomain.com \
  --region=asia-southeast1

# Add the CNAME record at your DNS provider:
# agent.yourdomain.com -> ghs.googlehosted.com

```

12.2 VPC Connector (Private Access)

```

# For agents that need to access private resources (Cloud SQL, Redis)
gcloud compute networks vpc-access connectors create agent-vpc \
  --region=asia-southeast1 \
  --range=10.8.0.0/28

gcloud run services update langgraph-agent \
  --region=asia-southeast1 \
  --vpc-connector=agent-vpc \
  --vpc-egress=private-ranges-only

```

13. Monitoring with Cloud Operations

Cloud Run Metrics (Automatic)

- Request count, latency (P50/P95/P99), error rate
- Container instance count, CPU/memory utilization
- Cold start frequency and duration
- Billable container instance time

Custom Metrics and Structured Logging

```
# Structured logging for Cloud Run
import json, sys

def log(severity, message, **kwargs):
    entry = {
        "severity": severity,
        "message": message,
        **kwargs
    }
    print(json.dumps(entry), file=sys.stdout, flush=True)

# Usage
log("INFO", "Agent request completed",
    tokens_used=1523, latency_ms=2340,
    model="gpt-4o", thread_id="abc123")

log("WARNING", "High token usage detected",
    tokens_used=45000, threshold=30000)
```

Set Up Alerts

```
# Alert on high error rate
gcloud monitoring policies create \
  --display-name="Agent Error Rate > 5%" \
  --condition-display-name="Cloud Run 5xx errors" \
  --condition-filter='resource.type="cloud_run_revision" AND \
    metric.type="run.googleapis.com/request_count" AND \
    metric.labels.response_code_class="5xx"' \
  --condition-threshold-value=5 \
  --condition-threshold-comparison=COMPARISON_GT \
  --notification-channels=CHANNEL_ID
```

14. Common Errors and Fixes

Error	Cause	Fix
Container failed to start	Wrong PORT (must be 8080)	Read PORT from os.environ; default 8080 for Cloud Run
Permission denied: Artifact Registry	Docker not authenticated	Run: gcloud auth configure-docker REGION-docker.pkg.dev
Cloud Run: Memory limit exceeded	Agent state too large	Increase --memory to 4Gi or 8Gi; limit message history

Error	Cause	Fix
Cloud Run: Request timeout	Agent chain takes > 300s	Increase --timeout (max 3600s); add streaming response
Secret not found	Secret Manager version issue	Use ':latest' suffix: secret-name:latest
Cold start > 10 seconds	Large Docker image	Use multi-stage build; enable --cpu-boost; set --min-instances=1
503 Service Unavailable	All instances busy or scaling	Increase --max-instances and reduce --concurrency
GKE Pod CrashLoopBackOff	App crashes repeatedly	Check logs: kubectl logs POD_NAME -n agentic-ai --previous
Permission denied: Secret Manager	Service account lacks access	Grant roles/secretmanager.secretAccessor to SA
Build failed: out of disk	Large build context	Add .dockerignore; exclude venv, .git, node_modules

15. GCP Deployment Options Comparison

Feature	Cloud Run	GKE Autopilot	GKE Standard	Compute Engine
Complexity	Very Low	Medium	High	Medium
Auto-scaling	Yes (0 to N)	Yes (pod-level)	Yes (pod + node)	Manual / MIG
Scale to zero	Yes	Yes (pods)	No	No
GPU support	No	Yes	Yes	Yes
Max request timeout	3600s (1 hour)	Unlimited	Unlimited	Unlimited
Cost model	Per-request + CPU-sec	Per-pod resources	Per-node + pod	Per-VM hour
Cold start	2-15 seconds	10-30 seconds	10-30 seconds	None (always on)
Best for	Most agent APIs	Multi-agent systems	Complex architectures	GPU/persistent workloads
Setup time	5 minutes	15 minutes	30 minutes	15 minutes
Networking	Built-in HTTPS	Ingress controller	Full control	Full control

Recommendation: Start with Cloud Run for 90% of agent deployments. Move to GKE only if you need multi-agent orchestration, GPU access, or custom networking. Use Compute Engine for persistent GPU workloads or background agent processes.